

LED MATRIX - 광안리의 밤

팀명: 흥박사와 아이들

팀원: 박규현, 임송주, 송성혁, 홍순현

1. 전체 시스템 개요

<코드 구조>

1.1 소스파일

- main.c
- panel.c
- fireworks.c
- bridge.c
- sound.c
- traffic.c

1.2 헤더파일

- main.h
- panel.h
- fireworks.h
- bridge.h
- sound.h
- traffic.h

1.3 패널 구동부 (panel.c)

:LED 패널을 1/16 스캔 방식과 4bit 밝기(BIT_DEPTH=4)로 구동한다.

내부에 비트플레인(framebuffer)을 가지고 있고, TIM2 인터럽트 기반으로 지속적인 스캔을 수행한다.

1.4 불꽃 애니메이션부 (fireworks.c)

:로켓 발사, 폭발, 파티클(불꽃 조각) 물리 시뮬레이션을 수행하고, 프레임버퍼에 그린 뒤 Panel_SetPixel로 패널에 전달한다. 자동 발사 모드, 특수 텍스트("CO-SHOW", "COSS", "POLARIS") 표시, 2단 폭발, 여러 폭발 모양(원, 링, 별, 하트)을 포함한다.

2. 패널 구동부 (panel.c) 설명

2.1 데이터 구조 및 전역 변수

```
static uint8_t planeR[BIT_DEPTH][PANEL_HEIGHT][PANEL_WIDTH];
static uint8_t planeG[BIT_DEPTH][PANEL_HEIGHT][PANEL_WIDTH];
static uint8_t planeB[BIT_DEPTH][PANEL_HEIGHT][PANEL_WIDTH];

static uint8_t curBit = 0;
static uint8_t curRow = 0;
static uint16_t ticksRemain = 0;
static uint8_t needLoad = 1;
```

각 색상에 대해 비트플레인 방식으로 프레임버퍼를 저장한다.

BIT_DEPTH(3bit)에 대해 bit 0~2을 나눠 저장함으로써 PWM 밝기 제어를 쉽게 한다.

curBit, curRow

-현재 어떤 비트플레인과 어떤 행(row group)을 스캔 중인지 나타내는 상태 변수이다.

ticksRemain, needLoad

-한 행/비트 상태를 얼마나 오래 출력할지(밝기 비율)를 관리하기 위한 변수이다.

-needLoad가 1이면 새로운 라인을 패널에 로드해야 함을 의미한다.

2.2 Panel_SetPixel()

```
void Panel_SetPixel(uint8_t x, uint8_t y, uint8_t r, uint8_t g, uint8_t b)
{
    if (x >= PANEL_WIDTH || y >= PANEL_HEIGHT) return;

    for (uint8_t bit = 0; bit < BIT_DEPTH; bit++) {
        planeR[bit][y][x] = (r >> bit) & 1;
        planeG[bit][y][x] = (g >> bit) & 1;
        planeB[bit][y][x] = (b >> bit) & 1;
    }
}
```

-애플리케이션 단에서 (x, y) 좌표의 픽셀 값을 3bit 비트플레인 형태로 프레임버퍼(planeR/G/B)에 기록한다.

-x, y 범위를 벗어나면 무시.

-각 bit(0~2)에 대해 (r >> bit) & 1 형태로 해당 비트에 대한 ON/OFF를 저장한다. G, B도 동일하게 처리한다.

불꽃 코드에서 0~7 범위의 색 값을 쓰면, 여기서 자동으로 PWM용 비트플레인으로 변환된다.

2.3 Panel_GPIO_Init()

```
void Panel_GPIO_Init(void)
{
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitTypeDef GPIO_InitStruct = {0};

    GPIO_InitStruct.Pin = PIN_R1|PIN_G1|PIN_B1|PIN_R2|PIN_G2|PIN_B2|PIN_CLK|PIN_LAT;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(PANEL_PORT_DATA, &GPIO_InitStruct);

    GPIO_InitStruct.Pin = PIN_A|PIN_B|PIN_C|PIN_D|PIN_OE;
    HAL_GPIO_Init(PANEL_PORT_CTRL, &GPIO_InitStruct);

    HAL_GPIO_WritePin(PANEL_PORT_CTRL, PIN_OE, GPIO_PIN_SET);
}
```

- 패널에서 사용하는 모든 GPIO 핀을 출력 모드로 초기화한다.
- GPIOA, GPIOB 클럭 활성화.
- 데이터/클럭/래치(PIN_R1, PIN_G1, ..., PIN_CLK, PIN_LAT)를 푸시풀 출력, 고속 속도로 설정.
- 행 선택 및 출력 Enable 핀(PIN_A, PIN_B, PIN_C, PIN_D, PIN_OE)도 동일하게 출력으로 설정.
- 초기 상태에서 PIN_OE를 SET(출력 차단)으로 두어 깜빡임을 방지.

2.4 내부 헬퍼 함수들

```
static inline void Panel_SetRow(uint8_t row)
{
    uint32_t odr = PANEL_PORT_CTRL->ODR & ~ROW_MASK;
    if (row & 1) odr |= PIN_A;
    if (row & 2) odr |= PIN_B;
    if (row & 4) odr |= PIN_C;
    if (row & 8) odr |= PIN_D;
    PANEL_PORT_CTRL->ODR = odr;
}
```

- : 스캔할 행 인덱스를 A,B,C,D 주소핀에 출력한다.
- 컨트롤 포트의 ODR에서 기존 row 비트들을 지우고, row의 하위 4bit를 A,B,C,D 핀에 매핑하여 설정한다.

OE_On(),OE_Off()

```
static inline void OE_On(void) { PANEL_PORT_CTRL->BSRR = (uint32_t)PIN_OE << 16; }
static inline void OE_Off(void) { PANEL_PORT_CTRL->BSRR = PIN_OE; }
static inline void CLK_Pulse(void)
{
    PANEL_PORT_DATA->BSRR = PIN_CLK;
    PANEL_PORT_DATA->BSRR = (uint32_t)PIN_CLK << 16;
}
static inline void LAT_Pulse(void)
{
    PANEL_PORT_DATA->BSRR = PIN_LAT;
    PANEL_PORT_DATA->BSRR = (uint32_t)PIN_LAT << 16;
}
```

-OE_On() : 출력 Enable(Active Low)을 활성화 → LED 켜짐.

-OE_Off() : 출력 비활성화 → LED 꺼짐.

-BSRR 레지스터를 직접 사용해 핀을 빠르게 On/Off 한다.

CLK_Pulse()

-shift 레지스터에 데이터를 한 비트 밀어 넣기 위한 클럭 펄스를 생성한다.

LAT_Pulse()

-shift된 데이터를 실제 출력 래치에 반영하기 위한 펄스를 생성한다.

2.5 Shift_Row()

```
static void Shift_Row(uint8_t bit, uint8_t rowGroup)
{
    uint8_t y1 = rowGroup;
    uint8_t y2 = rowGroup + PANEL_ROW_GROUPS;

    for (uint8_t x = 0; x < PANEL_WIDTH; x++) {
        uint32_t odr = PANEL_PORT_DATA->ODR & ~DATA_MASK;

        if (planeR[bit][y1][x]) odr |= PIN_R1;
        if (planeG[bit][y1][x]) odr |= PIN_G1;
        if (planeB[bit][y1][x]) odr |= PIN_B1;

        if (planeR[bit][y2][x]) odr |= PIN_R2;
        if (planeG[bit][y2][x]) odr |= PIN_G2;
        if (planeB[bit][y2][x]) odr |= PIN_B2;

        PANEL_PORT_DATA->ODR = odr;
        CLK_Pulse();
    }
}
```

-현재 bit plane과 row group에 해당하는 한 줄(상/하 2줄)을 시프트 레지스터로 밀어 넣는다.

-y1 = rowGroup, y2 = rowGroup + PANEL_ROW_GROUPS로 상·하 두 줄 인덱스를 계산.

-x=0~63까지 반복하면서 각 비트에 따라 R1/G1/B1, R2/G2/B2 라인을 세팅하고 CLK_Pulse() 호출.

-한 행(상/하 두 행)에 대한 RGB 데이터가 shift 레지스터에 모두 들어간다.

2.6 Panel_StartRefresh()

```
void Panel_StartRefresh(void)
{
    curBit = 0;
    curRow = 0;
    ticksRemain = 0;
    needLoad = 1;

    HAL_TIM_Base_Start_IT(&htim2);
}
```

-패널 스캔을 시작하기 위한 초기화 함수.

-curBit, curRow, ticksRemain, needLoad를 초기화하고,HAL_TIM_Base_Start_IT(&htim2)로 TIM2 인터럽트 시동. 이후 실제 스캔은 Panel_ScanISR에서 이루어진다.

2.7 Panel_ScanISR()

```
void Panel_ScanISR(void)
{
    const uint16_t baseTicks = 1; // 1,2,3 정도로 조절 가능 (밝기/프레임레이트 튜닝용)

    if (needLoad) {
        // 이전 라인 끄고 → 새 라인 데이터 로드
        OE_Off();
        Panel_SetRow(curRow);
        Shift_Row(curBit, curRow);
        LAT_Pulse();
        OE_On(); // 출력 거기

        ticksRemain = (uint16_t)(baseTicks << curBit); // 1,2,4,8 배 기준치
        needLoad = 0;
        return;
    }

    // 이 상태를 더 유지할지 확인
    if (ticksRemain > 0) {
        if (--ticksRemain == 0) {
            // 이 상태 끝 → 다음 상태로 넘어갈 준비
            needLoad = 1;

            // 다음 row / bit 결정
            curRow++;
            if (curRow >= PANEL_ROW_GROUPS) {
                curRow = 0;
                curBit++;
            }
        }
    }
}
```

-TIM2 인터럽트 핸들러에서 주기적으로 호출되는 스캔 루프의 핵심 함수.

needLoad == 1인 경우

-이전 라인을 끄고(OE_Off),

-현재 curRow를 A~D 핀에 세팅 후 Shift_Row(curBit, curRow)로 데이터 밀어넣고,

-LAT_Pulse()로 래치, OE_On()으로 출력 활성화.

-ticksRemain = baseTicks << curBit로 이 비트플레인의 유지 시간을 결정. (상위 비트일수록 더 오래 켜짐 → PWM)

-needLoad = 0으로 변경.

needLoad == 0인 경우

-ticksRemain을 감소시키다가 0이 되면, 다음 row로 이동(curRow++), 마지막 row까지 갔으

- 면 다음 bit plane(curBit++), 마지막 bit까지 끝나면 다시 0으로 돌아감.
- needLoad = 1로 바꾸어 다음 인터럽트에서 새 라인을 로드하도록 한다.
- 이 함수 하나로 4bit PWM + 1/16 스캔을 실시간으로 수행한다.
- 프레임버퍼를 바꾸더라도 스캔 로직은 그대로 유지된다.

3. 불꽃 애니메이션부 (fireworks.c) 설명

3.1 데이터 구조 및 전역 변수

: fireworks.c는 로켓·파티클·특수 텍스트 상태를 저장하고, 128×32 패널에 표시할 최종 프레임버퍼를 생성한다., 데이터 구조는 PDF 7~9 페이지의 서술 스타일을 그대로 따른다.

1) Rocket

- 위치 x, y
- 속도 vy
- 꼬리 색상 color_r/g/b
- alive, exploded 상태
- 폭발 모양(shape: 원, 링, 별, 하트 등)
- 특수 텍스트 타입(special_type: CO-SHOW, COSS, POLARIS)
- 생성 시각 birth_ms

2) Particle

- 좌표 x, y
- 속도 vx, vy
- 수명 life
- 감쇠 계수 decay
- 색상 color_r/g/b
- alive 여부

3) SpecialText

: panel.c의 planeR/G/B 비트플레인의 역할과 유사한 “최종 렌더링을 위한 중간 저장소”로 사용된다.

- 활성 여부(active)
- 텍스트 타입
- 위치 x, y
- 남은 수명 life
- 이 구조들은 화면에 그려지는 불꽃 애니메이션의 모든 상태를 표현하며

3.2 Fireworks_Init()

: panel.c의 Panel_StartRefresh()와 동일하게 프레임버퍼 비우기 → 상태 초기화 → 실행 준비” 형태를 따른다.

- 내부 128×32 프레임버퍼(fbR/fbG/fbB)를 모두 0으로 초기화
- 로켓·파티클·텍스트 배열 초기화
- 난수 시드 설정
- 자동 모드(auto_mode)와 다음 자동 발사 예약(next_auto_spawn_ms) 초기화

3.3 Fireworks_TriggerSpawn() (폭죽 발사)

```
void Fireworks_TriggerSpawn(void)
{
    float spawn_min_x = 2.0f;
    float spawn_max_x = (float)(VIRTUAL_W - 3); // 64쪽 기준 여유

    for (int i=0; i<MAX_ROCKETS; i++){
        if (!rockets[i].alive){
            rockets[i].alive = true;
            rockets[i].exploded = false;

            rockets[i].x = randf(spawn_min_x, spawn_max_x);

            // 물리적 아래 패널의 맨 아래(가산 Y=63)에서 위로 21픽셀 올라간 위치(Y=42)에서 시작
            rockets[i].y = (float)(VIRTUAL_H - 1 - 21); // 63 - 21 = 42

            rockets[i].vy = randf(-1.2f, -2.1f); // 위로 쏘아올림 (Y 감소 방향)

            // 로켓 기본 꼬리 색
            int c = rand() % 4;
            switch (c) {
                case 0: rockets[i].color_r = 15; rockets[i].color_g = 6; rockets[i].color_b = 0; break;
                case 1: rockets[i].color_r = 12; rockets[i].color_g = 0; rockets[i].color_b = 12; break;
                case 2: rockets[i].color_r = 0; rockets[i].color_g = 12; rockets[i].color_b = 12; break;
                default: rockets[i].color_r = 12; rockets[i].color_g = 12; rockets[i].color_b = 3; break;
            }

            rockets[i].birth_ms = HAL_GetTick();
            rockets[i].special_type = ((rand() % 3) == 0) ? (rand() % SPECIAL_TYPES) : -1;
            rockets[i].shape = (ExplosionShape)(rand() % 4);

            break;
        }
    }
}
```

: 패널 한 줄을 shift하여 스캔하는 panel.c의 Shift_Row()처럼 이 함수는 “새 폭죽 하나를 시뮬레이터에 밀어 넣는” 역할을 한다.

-동작 과정

>비활성 로켓 슬롯 탐색 ->초기 위치 설정 (x: 2~61, y: 42 부근) ->위로 상승하는 초기 속도 설정 ->꼬리 색상 팔레트 선택 ->폭발 모양(shape) 난수 배정 ->일정 확률로 특수 텍스트 활성화

3.4 Fireworks_ToggleAuto() (자동 발사 모드)

```
void Fireworks_ToggleAuto(void)
{
    auto_mode = !auto_mode;

    if (auto_mode) {
        uint32_t now = HAL_GetTick();
        next_auto_spawn_ms = now + random_interval_ms();
    } else {
        next_auto_spawn_ms = 0;
    }
}
```

: 버튼 인터럽트에서 호출되며 자동 모드를 토글한다.

- auto_mode = 1 → 자동 발사 예약
- auto_mode = 0 → 예약 해제

3.5 Fireworks_Update(now)

: 패널 스캔 루프(Panel_ScanISR)가 시간 기반 LED PWM을 수행하는 것처럼 이 함수는 애니메이션의 시간 기반 물리 시뮬레이션을 수행한다.

-동작 과정

> Δt 계산->모든 로켓 이동->임계점 도달 시 폭발 처리 ->파티클 이동, 감쇠, 수명 감소

->일부 파티클은 수명 종료 시 2차 폭발 ->자동 모드일 경우 예약 시간 도달 시

Fireworks_TriggerSpawn() 호출

3.6 Fireworks_Render()

: panel.c의 Panel_SetPixel() 과정처럼 fireworks.c는 프레임버퍼를 생성해 패널 구동부로 전달한다.

-렌더링 과정

1)배경 이미지 복사

-bridge.c의 img_128x32를 전체 프레임에 복사

2) 64×64 가상 공간 → 128×32 실 공간 매핑

-가상 상단(0~31) / 하단(32~63) 두 공간을 물리 패널 상·하 64×32로 매핑한다.

3) 로켓/파티클 색상 누적(Additive Blending)

-밝기(intensity)를 더하는 방식으로 그려자연스러운 잔광 효과와 중첩 표현을 구현한다. 결과는 4bit(0~15)로 클램프하여 panel.c 비트플레인과 호환한다.

4. 다리 구현 (bridge.c) 설명

4.1 img_128x32

: 플래시 공간에 고정된 원시 이미지 버퍼로 사용된다. 각 픽셀 값의 상위 4비트가 R, 중간 4비트가 G, 하위 4비트가 B를 의미한다. 이 테이블은 패널 스캔 루틴이 읽어갈 기본 배경 그림 역할을 한다.

4.2 Bridge_Init()

: 브리지 모듈의 선행 초기화 지점을 제공한다.

4.3 Bridge_GetPixel()

```
void Bridge_GetPixel(int x, int y, uint8_t *r, uint8_t *g, uint8_t *b)
{
    if (x < 0 || x >= 128 || y < 0 || y >= 32) {
        *r = *g = *b = 0;
        return;
    }

    uint16_t v = img_128x32[y][x];

    uint8_t r4 = (v >> 8) & 0x0F;
    uint8_t g4 = (v >> 4) & 0x0F;
    uint8_t b4 = (v >> 0) & 0x0F;

    *r = r4;
    *g = g4;
    *b = b4;

    Traffic_OverlayPixel(x,y,r,g,b);
}
```

: 배경 이미지에서 픽셀을 꺼내고, 선택적 오버레이를 적용하여 최종 색을 돌려주는 함수다. 경계 검사로 좌표 유효성을 확보하고, 0xRGB 포맷을 비트 시프트·마스크로 분해하여 채널을 추출한다. 추출된 값은 4비트 선형 밝기로 패널의 비트플레인에 바로 매핑할 수 있도록 전달된다. 마지막에 Traffic_OverlayPixel()을 호출하여 신호·문구 같은 동적 요소를 배경 위에 합성되도록 한다.

5. 소리 구현 (sound.c) 설명

5.1 헤더/외부 선언 영역

```
#include "stm32f4xx_hal.h"
#include "sound.h"
#include <stdlib.h>
// main.c 어딘가에 TIM3 핸들 만들어줄 거라 extern
extern TIM_HandleTypeDef htim3;

// 타이머 tick = 1MHz (PSC=83 가정)
#define BUZZER_TIMER_HZ 1000000UL

// 피음(상승음) + 퍼버백(폭발 노이즈) 시간
#define FW_RISE_TIME_MS 180U
#define FW_EXPLODE_TIME_MS 150U
```

HAL 드라이버, 사운드 모듈 인터페이스, rand()를 쓰기 위한 표준 라이브러리를 포함한다. TIM3 PWM을 제어할 htim3를 외부에서 가져와 이 파일에서 사용한다.

5.2 타이머/효과 파라미터 상수

```
// 피음(상승음) + 퍼버백(폭발 노이즈) 시간
#define FW_RISE_TIME_MS 180U
#define FW_EXPLODE_TIME_MS 150U

// 피음 주파수 범위
#define FW_F_START_HZ 800U // 시작(저음)
#define FW_F_END_HZ 3500U // 끝(고음)
```

:TIM3의 내부 카운터가 1MHz로 동작한다고 가정하고 주파수 계산에 사용한다.

상승음 구간과 폭발 구간의 지속 시간을 밀리초로 정의한다.

상승음의 시작·끝 주파수를 정의해 선형 보간의 구간을 정한다.

5.3 상태 머신과 내부 시간 변수

사운드 재생 흐름을 IDLE → RISE → EXPLODE의 3단계 상태로 관리한다.

각 단계의 시작 시각과 마지막 파라미터 갱신 시각을 저장해 시간 기반 업데이트를 가능하게 한다.

5.4 로우레벨 PWM 제어 함수들

요구한 주파수에 맞는 ARR 값을 계산하고 16비트 범위로 클램프하여 적용한다.

기본 듀티를 50%로 잡고 CCR에 기록해 즉시 음 높이가 반영되게 한다.

카운터를 0으로 리셋해 위상 일관성을 확보한다.

5.5 공개 API: 초기화

```
void Sound_Init(void)
{
    // TIM3 PWM은 MX_TIM3_Init에서 이미 설정된 상태라고 가정
    // 여기서는 PWM만 시작 + duty 0으로 mute
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4);
    buzzer_stop();

    s_state = SOUND_IDLE;
}
```

TIM3 채널4 PWM 출력을 시작하고 듀티 0%로 음소거 상태에서 대기한다.

상태를 IDLE로 두어 외부 트리거가 들어올 때까지 멈춰있게 한다.

5.6 공개 API: 재생 트리거

```
void Sound_PlayFirework(void)
{
    // 이미 다른 효과 재생 중이면 덮어쓰기 (재시작)
    s_state = SOUND_RISE;
    s_phase_start_ms = HAL_GetTick();
    s_last_step_ms = s_phase_start_ms;

    // 초기 freq/duty 설정
    buzzer_set_freq(FW_F_START_HZ);
    buzzer_set_duty_percent(40); // 40% 정도
}
```

호출 순간 상승음 단계로 전환하고 타이밍 기준을 현재 시각으로 초기화한다.

시작 주파수와 적당한 듀티로 초기 세팅하여 즉시 피치가 올라가도록 준비한다.

5.7 상태 업데이트 루프

대기 상태에서는 아무 작업도 하지 않고 빠져나간다.

상승음 단계에서는 10ms 주기로 진행도 t를 계산해 시작~끝 주파수 사이를 선형 보간한다.

계산된 주파수와 약간 올린 듀티를 반영해 시간이 지날수록 음이 높아지게 만든다.

10ms 경과 조건으로 상태 전환과 파라미터 갱신 타이밍을 제어한다. 폭발 단계에서는 목표 시간까지 유지하며 8ms 주기로 랜덤 주파수와 듀티를 갱신한다.

1/5 확률로 완전 무음을 삽입해 끊기는 질감을 만들어 폭발 노이즈 느낌을 강화한다.

단계 시간이 끝나면 즉시 음소거하고 IDLE로 복귀한다.

6. 메인 실행부 (main.c) 설명

6.1 초기화 함수 구성

: main.c는 전체 시스템의 초기화와 주 반복 루프를 담당한다. 프로그램 시작 시 HAL 및 시스템 클럭 초기화, GPIO·타이머 설정, 패널 구동부 시작, 배경 이미지/불꽃 엔진/사운드 모듈 초기화가 순차적으로 호출된다.

1) HAL_Init(), SystemClock_Config()

-MCU 기본 환경 및 클럭 설정을 완료한다.

2) MX_GPIO_Init(), MX_TIM2_Init(), MX_TIM3_Init()

-LED 패널 스캔을 위한 TIM2 인터럽트, 사운드 PWM을 위한 TIM3 설정을 완료한다.

3) Panel_GPIO_Init(), Panel_StartRefresh()

-패널 구동부(panel.c)에서 정의된 GPIO 세팅 및 스캔 시작.

-TIM2 인터럽트 기반의 1/16 스캔 루틴이 여기서 시동된다.

4) Bridge_Init()

-배경 이미지 모듈 사전 초기화.

5) Fireworks_Init()

-모든 로켓, 파티클, 텍스트 상태를 초기화하고 내부 난수 시드를 설정한다.

6) Sound_Init()

-TIM3 PWM을 0% 듀티로 시작하여 사운드 모듈을 대기 상태로 둔다.

```
Panel_GPIO_Init();
Panel_StartRefresh();
Bridge_Init();
Fireworks_Init();
Sound_Init();
```

6.2 외부 인터럽트(EXTI) 처리

: main.c는 두 개의 사용자 입력 버튼을 EXTI 인터럽트로 처리한다.

-PB0 : 수동 폭죽 발사 버튼
150ms 디바운싱 후 Fireworks_TriggerSpawn() 호출

-PB4 : 자동 발사 모드 토글 버튼
200ms 디바운싱 후 Fireworks_ToggleAuto() 호출

6.3 메인 루프(Main Loop)

```
while (1)
{
    uint32_t now = HAL_GetTick();
    Fireworks_Update(now);
    Sound_Update(now);
    // 렌더는 UPDATE_INTERVAL_MS 주기보다 조금 더 자주 해도 무방, 여기서 30ms 이내로
    if (now - last_render >= 30) {
        Fireworks_Render();
        last_render = now;
    }
    // 다른 작업이 있다면 여기서 처리
    // 가벼운 슬립
    HAL_Delay(2);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
```

: 초기 설정이 끝나면 main.c는 무한 반복 루프에서 불꽃 애니메이션과 사운드를 주기적으로 갱신한다.

-갱신 구성

Fireworks_Update(now)

-로켓/파티클의 물리 기반 위치 업데이트
-폭발 판정, 수명 감소, 2단 폭발 처리

Sound_Update(now)

-불꽃 상승음·폭발음 상태 머신 처리

7. 교통 애니메이션부 (traffic.c) 설명

: traffic.c는 하단 패널(64~127번 X 좌표)에 위치한 도로 위를 일정 간격으로 자동차가 지나가는 효과를 구현한다. 자동차는 최대 8대까지 동시에 관리되며, 각 차량은 x, y 좌표와 활성 상태(active)만으로 단순하게 표현된다.

7-1 W, H: 패널 전체 크기(128×32)

BOTTOM_PANEL_START_X = 64

상단 패널(0~63)과 하단 패널(64~127)을 구분하기 위한 기준 값

7-2 ROAD_Y

하단 패널 기준에서 위로 24픽셀 위치한 도로 라인

7-3 MAX_CARS = 8, CAR_LEN_PIXELS = 2

자동차 개수와 자동차의 픽셀 길이

7-4 시간 변수

last_step_ms : 차량 이동 간격을 제어

last_spawn_ms : 차량 생성 간격을 제어

next_spawn_interval_ms : 다음 생성까지의 랜덤 시간(4.5~6초)

7-5 헬퍼 함수

1) has_car_near_entry()

```
// 오른쪽 끝(하단 패널의 입구) 근처에 차가 있는지 확인
static bool has_car_near_entry(void)
{
    for (int i = 0; i < MAX_CARS; i++) {
        if (!cars[i].active) continue;
        // 화면 오른쪽 끝(128)에서 6픽셀 이내에 차가 있으면 스폰 금지
        if (cars[i].x > W - 6) {
            return true;
        }
    }
    return false;
}
```

: 하단 패널의 오른쪽 끝(128번 근처)에 차량이 이미 존재하면 새로운 차량이 겹쳐 생성되는 것을 방지하기 위해 확인한다. x > 122 영역에 차량이 있으면 true를 반환한다.

2) spawn_car()

```
static void spawn_car(void)
{
    int idx = -1;
    for (int i = 0; i < MAX_CARS; i++) {
        if (!cars[i].active) {
            idx = i;
            break;
        }
    }
    if (idx < 0) return;

    if (has_car_near_entry()) return;

    // 하단 패널의 오른쪽 끝(128)에서 시작
    cars[idx].x = W;
    cars[idx].y = ROAD_Y;
    cars[idx].active = true;
}
```

: 새로운 차량을 생성하는 함수이다.

-비활성화된 차량 슬롯 검색

-has_car_near_entry() 결과가 true이면 생성 중단

-생성 위치 x = 128, y = ROAD_Y

-active = true 로 설정

자동차는 항상 하단 패널의 가장 오른쪽에서 등장하도록 고정된다.

7-6 Traffic_Init()

```
static void spawn_car(void)
{
    int idx = -1;
    for (int i = 0; i < MAX_CARS; i++) {
        if (!cars[i].active) {
            idx = i;
            break;
        }
    }
    if (idx < 0) return;

    if (has_car_near_entry()) return;

    // 하단 패널의 오른쪽 끝(128)에서 시작
    cars[idx].x = W;
    cars[idx].y = ROAD_Y;
    cars[idx].active = true;
}
```

: 모든 차량을 비활성화로 초기화, 자동차 등장 빈도를 일정 수준으로 조절하여, 지나치게 바빠지 않고 자연스러운 흐름을 만든다.

- last_step_ms, last_spawn_ms를 현재 시각(now)으로 설정
- 다음 자동차 생성 시간(next_spawn_interval_ms)을
- 4.5~6초 사이의 랜덤 값으로 설정

7-7 Traffic_Update()

: Traffic_Update()는 “이동”과 “생성” 두 부분으로 이루어진다.

1) 이동 처리

- 80ms(CAR_STEP_INTERVAL_MS)마다 모든 활성 차량의 x좌표를 1픽셀씩 감소
- 차량이 하단 패널 영역을 완전히 벗어나상단 패널(0~63번 X 영역)으로 들어가면 즉시 비활성화 → 화면에서 자연스럽게 사라진다

2) 생성 처리

- (now - last_spawn_ms) ≥ next_spawn_interval_ms 일 경우 새로운 차량 생성
- 생성 후 next_spawn_interval_ms를 다시 랜덤(4.5~6초)으로 갱신

7.8 Traffic_OverlayPixel(x, y, r, g, b)

```
void Traffic_OverlayPixel(int x, int y, uint8_t *r, uint8_t *g, uint8_t *b)
{
    // 1. 도로 라인에 아니면 무시
    if (y != ROAD_Y) return;

    // 2. 상단 패널 영역(0~63)이면 무시 (하단 패널은 64~127만 그리기)
    if (x < BOTTOM_PANEL_START_X) return;

    for (int i = 0; i < MAX_CARS; i++) {
        if (!cars[i].active) continue;

        int cx = cars[i].x;
        int cx_end = cx + CAR_LEN_PIXELS;

        if (x >= cx && x < cx_end) {
            // 자동차 그리기
            *r = CAR_COLOR_R;
            *g = CAR_COLOR_G;
            *b = CAR_COLOR_B;
            return;
        }
    }
}
```

